

Is the browser the bottleneck? — AI-DLPP capability-ceiling study, executive summary

Policy-conditioned, fully-local detection of confidential data in LLM prompts · shippable in-browser 2–4 B models vs the same method on hosted open-weight models of 27–120 B where the size is published · 6,502 calls, three passes at temperature 0, every figure regenerated from artifacts · draft v1, 8 Sep 2026

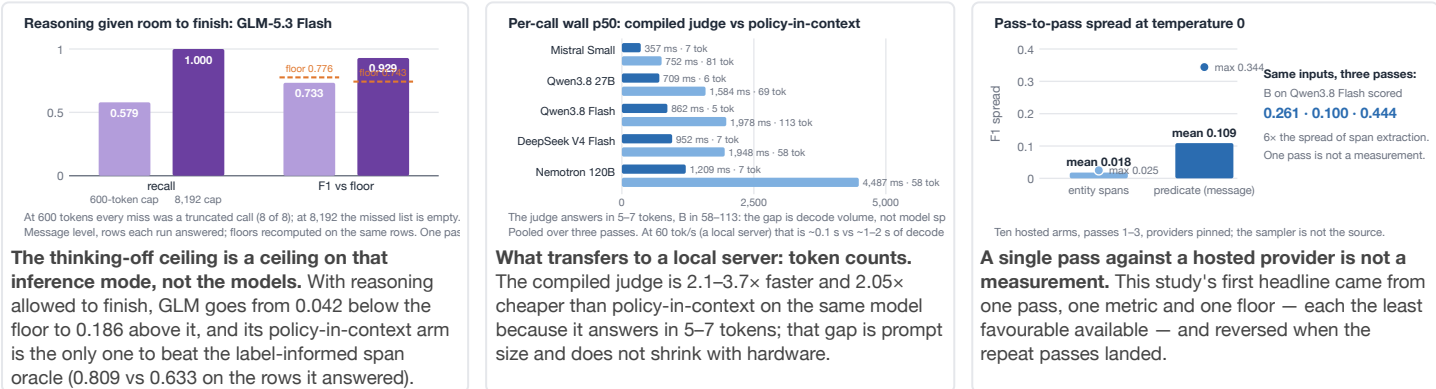
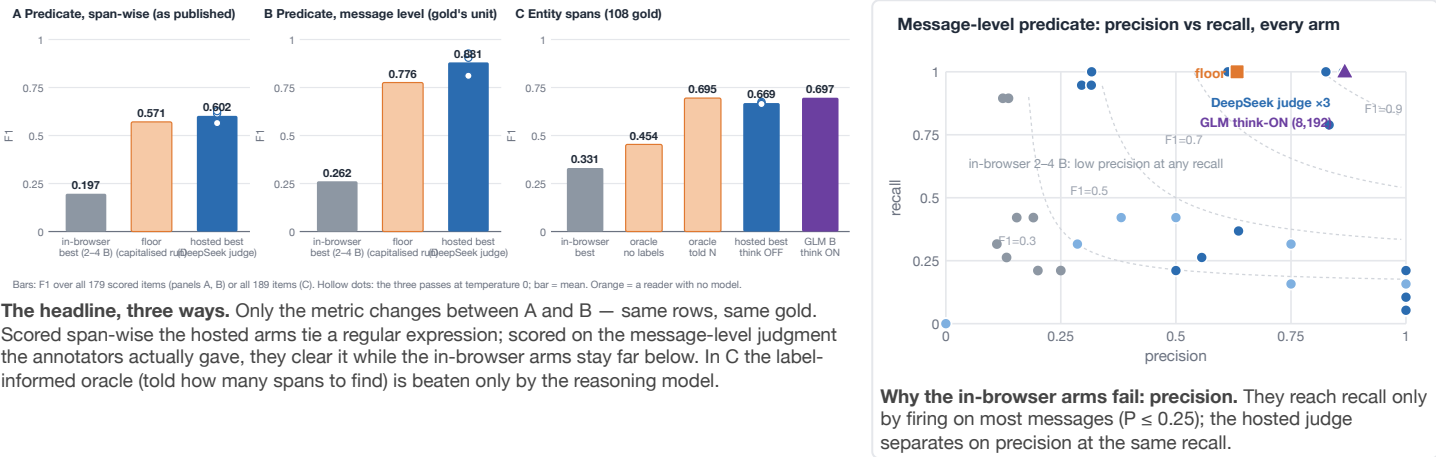
Question. Our in-browser detectors find the right region of a prompt and attach the wrong label. Is that the browser's constraint, or the task's difficulty? **Method.** Run the identical compiled judge and policy-in-context baseline on six much larger hosted models, providers pinned, three passes, and print a model-free floor beside every number.

0.262 → 0.881
message-level F1 at the policy's predicate, best in-browser vs best hosted arm. Floor **0.776**; all three hosted passes clear it.

6×
pass-to-pass variance at temperature 0 for classification (0.109) vs span extraction (0.018): unstable exactly where inaccurate.

0.579 → 1.000
recall of the mandatory-reasoning model when its cap rises from 600 to 8,192 tokens. Every miss at 600 was a truncated call.

\$1.22 · 0 / 5,580
total billed, and malformed JSON responses from provider-side constrained decoding in 5,580 thinking-off calls.



Conclusion

- The browser constraint is a real part of what costs accuracy.** On the judgment the gold records, models 6–60× larger than the shippable ones go from far below a trivial floor to clearly and repeatably above it (+0.105 mean over three passes); on the stricter span-matched reading they tie it. Both are true; the original "scale stops at a regex" rested on the stricter reading alone.
- The binding constraint at every scale is classification, not span extraction.** Spans are placed byte-exactly by a 2 B model and resolved cleanly 99.2% of the time at 27–120 B; what moves between passes, and what separates arms, is the label.
- Carry the compiled judge to a local server, alongside policy-in-context for entities.** The judge wins the semantic predicate (0.927 vs 0.457) and is cheaper and faster; policy-in-context wins entity finding (0.855 vs an always-fire floor of 0.727). A deployment runs both — which the tiered design already does.
- Reasoning buys accuracy at a latency price this corpus makes visible** — 31.8 minutes for 189 messages on the B family — and at 600 tokens the token cap, not the model, was the limit.

Verdict: the task is learnable by open-weight models a company could host; the shippable in-browser tier is not yet capable of it; the gap is precision on the semantic predicate, and it is substantially the price of the in-browser constraint.

Bounds: one policy (p-fin), one predicate, 19 gold positives (one item ≈ 0.03 F1 for the headline arm); the corpus's known generator leak gives the capitalisation floor perfect recall, so no floor figure transfers to a corpus without it; the thinking-ON result is one model, one pass, with 31% of rows lost to provider rate limiting; hosted quantisation is the provider's. Six defects in the study's own instrumentation are reported in the paper (§5). Repository [orkancelikhisar/ai-dlpp](#), docs/paper/; full records in docs/research/.